

MAC0209 — Modelagem e Simulação

Exercício-Programa 5 – Convolução

Grupo Pace Perfeito

Derek Ferreira Queiroz

Ezequiel Gomes de Carvalho

Leonardo Fraga Odloak

Lucas Pires da Silva

Luis Felipe Quireza Menezes de Oliveira

Matheus de Oliveira Teodoro da Silva

Milena de Paula Silva

Sandro Kenvi Mayorga Cáceres

Bacharelado em Ciência da Computação

IME – USP

Contexto do curso

- Na última aula, iniciamos com as apresentações sobre o EP 4, que consistiu em construir mapas logísticos e um mapa de bifurcação (tendo como base $x_{n+1} = 4rx_n(1 - x_n)$). Depois disso, tivemos uma aula sobre sistemas dinâmicos discretos.

Objetivo do EP

- Nesse EP, falaremos sobre convolução, que é uma forma de descrever como a entrada e as características do sistema interagem para produzir a saída quando tratamos de processamento linear de sinais.

Conceitos e implementação

- A convolução discreta pode ser definida como

$$(x * h)(n) = \sum_{i=-\infty}^{\infty} x(i) \cdot h(n - i)$$

Sendo $x(n)$ o sinal e $h(n)$ o filtro

- Nesse EP, testamos a convolução em sinais de 1 e 2 dimensões, utilizando a função `convolve` implementada pelo grupo.
- Fizemos vários testes; as imagens a seguir são, respectivamente:
 - Sinal 1d de números aleatórios com filtro $[-1, 0, 1]$
 - Sinal 2d (imagem) com o filtro sendo uma sequência de $\frac{1}{60}$ (sendo 60 o tamanho do filtro)
 - Sinal 2d com o filtro sendo uma sequência de números aleatórios entre -1 e 1
 - os códigos

Resultados

Sinal Original

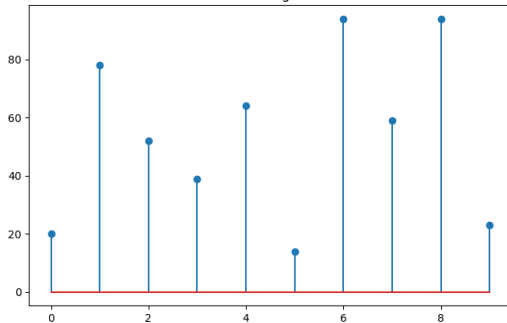


Figura: Sinal Original 1D

Sinal Filtrado

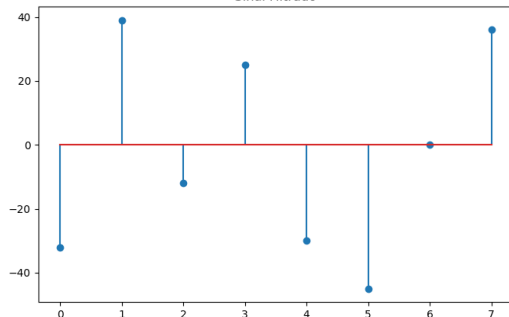


Figura: Sinal 1D Filtrado



Figura: Filtro com 1/60

```
def convolve(x_n, h_n):
    y = []
    for j in range(len(x_n) - len(h_n)):
        valor = 0
        for i in range(len(h_n)):
            valor += x_n[j + i] * h_n[i]
        y.append(valor)
    return y
```

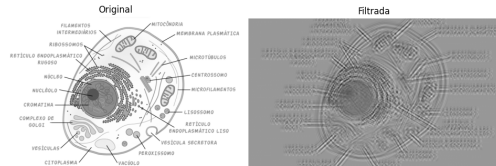


Figura: Aleatórios

```
def convolucao_linhas(img, filtro):
    h, w = img.shape
    out_w = w - len(filtro) + 1
    out = np.zeros((h, out_w))

    for i in range(h):
        out[i, :] = np.convolve(img[i, :], filtro, mode="valid")

    return out

def testar_filtro(filtro, nome, img, tamanho_filtro):
    horiz = convolucao_linhas(img, filtro)

    result_n = convolucao_linhas(horiz.T, filtro).T

    print("Imagem:", nome)
    print("Shape original:", img.shape)

    return result_n
```